Департамент
образования и науки
города Москвы

Государственное бюджетное
профессиональное образовательное
учреждение города Москвы

Колледж информационных
технологий «ИТ.МОСКВА»

IT.МОСКВА

8 499 612-03-25
spo-mgkeit@edu.mos.ru

115446, г. Москва,
ул. Академика Миллионщикова, д. 20

00216071 ОКПО
1027739491790 ОГРН
7724035635 ИНН
772401001 КПП

ОТЧЁТ
по учебной практике

Специальность 09.02.07 Информационные системы и Программирование (код, наименование)

Вид практики Учебная практика УП.02

Сроки прохождения практики 29.09.26 - 12.10.26

Место прохождения практики колледж ИТ.МОСКВА
(название предприятия)

Выполнил студент третьего курса группы **3-ИП-7-24**

Фролов Илья Сергеевич
(Ф.И.О. студента)

Москва, 2026

Занятие 1. Определение границ учебной ОС my_os

Задача

Определить границы учебной операционной системы my_os, разработать сценарии использования, сформулировать нефункциональные требования с метриками.

Что сделано

Спроектирована учебная ОС my_os с монолитным ядром. В качестве носителя данных — SQLite, работа с аппаратным обеспечением заменена программными заглушками. Акцент — на подсистеме планирования процессов.

В границы ОС входят:

- управление процессами;
- управление памятью;
- файловая система на SQLite;
- разграничение прав пользователей (SHA-256);
- командная оболочка (shell);
- подсистема журналирования;
- планировщик процессов (FCFS, Round Robin, приоритетный).

Не входят: работа с реальным железом, прерывания и драйверы, сетевые протоколы, GUI, интеграция с LDAP/AD.

Разработано 11 сценариев использования: вход в систему, создание/чтение файла, запуск процесса, просмотр списка процессов, завершение процесса, просмотр журнала, выделение памяти, завершение работы ОС, постановка в очередь, смена приоритета.

Нефункциональные требования:

Категория	Требование	Метрика
Производительность	Обработка системного вызова	≤ 50 мс
Производительнос ть Надёжность	Одновременных процессов Перехват исключений	≥ 10 100 %
Безопасность	Пароли в SHA-256	Отсутствие открытых паролей
Поддерживаемость	Модуль кода	≤ 300 строк
Ресурсоёмкость	Библиотеки	Только встроенные

Проанализировано 9 рисков (нехватка времени, блокировки SQLite, сложность отладки, голодание процессов и др.) с указанием стратегий минимизации.

Созданные файлы

Модуль	Назначение
src/db.py	Подключение к SQLite, инициализация таблиц users, processes, files, syscalls_log

src/syscalls.py	Реализация sys_echo, sys_get_users, логирование

src/shell.py	Командная оболочка: help, echo, users, exit

src/kernel.py	Ядро (заготовка)
src/config.py	Конфигурация: путь к БД, лимиты

Вывод

Границы ОС определены, разработано 11 сценариев, сформулированы нефункциональные требования, проведён анализ рисков, создан базовый прототип.

Занятие 2. Архитектурное проектирование и API ядра

Задача

Спроектировать модульную архитектуру ядра, разработать API системных вызовов, реализовать заглушки и логирование.

Что сделано

ОС переименована в StudyOS. Спроектирована модульная архитектура:

- shell.py — командная оболочка;
- syscalls.py — реализация вызовов и логирования;
- kernel.py — ядро (обработка вызовов, проверка прав);
- scheduler.py — планировщик (FCFS, Round Robin, приоритетный); ● memory.py — управление памятью;
- fs.py — файловая система;
- auth.py — аутентификация;
- db.py — подключение к SQLite;
- config.py — конфигурация.

Направление вызовов: **shell.py → syscalls.py → kernel.py → scheduler / memory / fs / auth → db.py**. Оболочка не обращается к БД напрямую.

Реализация API

В файле syscalls.py реализована функция логирования log_syscall() и **13 заглушек** системных вызовов: sys_login, sys_logout, sys_whoami, sys_create_file, sys_read_file, sys_delete_file, sys_list_files, sys_exec и другие. Каждая функция пишет запись в таблицу syscalls_log.

Проверка логирования

После запуска **python -m src.syscalls** в таблице syscalls_log появились 4 записи:

syscall_name	args	user	status

sys_login	admin	admin	OK
sys_whoami		admin	OK
sys_create_file	/test.txt	admin	OK

sys_ps		admin	OK

Обновление оболочки

В shell.py добавлены команды: whoami, login, logout, create, ls, ps, logs.

Вывод

Спроектирована модульная архитектура StudyOS, разработан API из 13 вызовов, реализовано логирование, обновлена оболочка. Обеспечено разделение ответственности: shell не работает с БД напрямую.

Практическая работа №3. Построение архитектуры программного средства

Задача

Построить архитектуру ИС «Кадровое агентство»: эскизный и технический проект, диаграммы, структурные карты.

Предметная область

Кадровое агентство ведёт учёт безработных и вакансий, подбирает варианты трудоустройства, оформляет направления, фиксирует трудоустройство, формирует отчётность для налоговых органов и госстатистики.

Функциональные требования

1. Ведение базы резюме безработных.
2. Ведение базы вакансий от предприятий.
3. Классификация резюме и вакансий по категориям.
4. Автоматический подбор вакансий под резюме.
5. Оформление направления на вакансию.
6. Фиксация трудоустройства.
7. Формирование отчётности.
8. Разграничение прав доступа (администратор, инспектор, бухгалтер). 9. Поиск и фильтрация данных.
10. Хранение истории действий.

Эксплуатационные требования

Параметр	Значение
Надёжность	Восстановление ≤ 1 час, ежедневное резервное копирование

Условия	Офис, 18–25 °C, влажность 40–70 %
Сервер	4 ядра, 16 ГБ ОЗУ, SSD 500 ГБ
Рабочие станции	2 ядра, 8 ГБ ОЗУ, SSD 256 ГБ
ПО	Windows 10/11, Ubuntu Server 22.04, PostgreSQL 15, Python 3.11

Выбор технологий

- **Метод:** структурный подход, нисходящее проектирование.
- **Язык:** Python 3.11.
- **Фреймворк:** Django (веб-приложение).
- **СУБД:** PostgreSQL 15.
- **Средства:** MS Visio, draw.io, MS Office.

Структура ПО

1. Подсистема авторизации и разграничения доступа.
2. Подсистема учёта безработных.
3. Подсистема учёта вакансий.
4. Подсистема подбора и трудоустройства.
5. Подсистема отчётности.
6. Подсистема администрирования.

Основные функции

- F1 — добавление/редактирование/удаление резюме.
- F2 — добавление/редактирование/удаление вакансии.
- F3 — поиск и фильтрация.
- F4 — автоматический подбор вакансий.
- F5 — оформление направления.
- F6 — фиксация трудоустройства.
- F7 — формирование отчётов.
- F8 — управление пользователями.

Спецификации процессов

Описаны 4 процесса: «Добавление резюме», «Подбор вакансии», «Оформление трудоустройства», «Формирование отчёта». Для каждого указаны вход, выход и алгоритм.

Диаграммы

- **DFD уровня 0:** внешние сущности — соискатель, предприятие, налоговые органы, госстатистика.
- **DFD уровня 1:** процессы P1–P4, хранилища D1–D4.
- **DFD уровня 2:** декомпозиция P3 на P3.1, P3.2, P3.3.
- **ERD:** сущности Соискатель, Резюме, Предприятие, Вакансия, Направление, Трудоустройство, Пользователь, Отчёт.

Технический проект

Разработаны: структурная схема (6 подсистем), функциональная схема (таблица «компонент-вход-выход»), алгоритм программы, структурная карта Константайна, структурная карта Джексона, технический проект модуля «Подбор и трудоустройство».

Вывод

Построена архитектура ИС «Кадровое агентство»: выполнены анализ требований, эскизный и технический проекты, построены DFD, ERD, структурные карты.

Практическая работа №4. Система контроля версий + ООП

Задача

Разработать программу на Visual Basic с ООП-подходом (автосалон «START») и изучить работу в Git.

Разработка класса Class1

Создан класс Class1 с полями:

Поле	Тип	Назначение
avto	Boolean	Флаг согласия на покупку
firma	String	Фирма-изготовитель
model	String	Марка автомобиля
stoim	String	Стоимость в долларах
pict	String	Номер PictureBox
var	String	Режим вывода

Методы класса:

- Met1() — проверяет согласие, вызывает Met2, Met3, Met4;
- Met2() — определяет марку по фирме;
- Met3() — определяет стоимость;
- Met4() — определяет номер PictureBox;
- Property Get varian() и Property Let varian() — свойства.

В Met2 определены 13 фирм (AUDI, CITROEN, FORD, HONDA, HYUNDAI, JEEP, LAND ROVER, LEXUS, MITSUBISHI, NISSAN, PEUGEOT, PORSCHE, RENAULT).

Разработка формы Form1

Элемент	Имя Назначение
ComboBox	Combo1 Выбор фирмы

ComboBox CommandButton	Combo2 Выбор режима вывода Command1, Command2 OK, Exit
Label	Label1–Label5 Надписи
OptionButton	Option1(0), Option1(1) Да / Нет

PictureBox Picture1(0..12), Picture2(0..12) Фото и характеристики

Реализованы обработчики событий: Combo1_Click, Combo2_Click, Command1_Click, Command2_Click, Form_Load.

Работа в Git

Выполнены команды:

```
git init git config --global user.name "Student" git add .
git commit -m "Initial commit: класс Class1 и форма Form1"
   git branch feature-add-toyota git checkout feature-add-toyota git commit-m
```

"Добавлена

фирмаTOYOTA"gitcheckoutmaingitmergefeature-add-toyotagitlog--oneline

--graph --all

Результат

Разработан класс Class1 и форма Form1, освоен Git: init, add, commit, branch, merge, log. Создана и слита feature-ветка.

Вывод

Изучен ООП на примере автосалона, освоена работа в Git (создание репозитория, ветвление, слияние, просмотр истории).

Практическая работа №5. Основы Python. Чудо Обувь

Задача

Освоить базовые конструкции Python и структуры данных на примере предметной области «Чудо Обувь».

Что изучено

- Переменные и типы: int, float, str, bool.
- Операторы: +, -, *, /, //.
- Условия: if / elif / else.
- Циклы: for, while, break, continue.
- Коллекции: list, dict, tuple.
- Функции (def), return.
- Сортировка через sorted и lambda.
- Фильтрация через списковые включения.
- f-strings, встроенные функции len, sum, min, max.

Реализовано 8 заданий

1. Первая программа (print).
2. Переменные, print, for.
3. Списки и словари (users, products).
4. Функции find_user и total_sum.
5. Сортировка и фильтрация.
6. Поиск по каталогу (search_products, filter_by_price).
7. Мини-задача «Корзина» (add_to_cart, remove_from_cart, change_quantity).
8. Самостоятельная работа: добавлены 2 товара, get_sizes, get_total_quantity, сортировка, фильтрация, вывод через while.

Дополнительно

Смоделированы заказы: список orders, сортировка по дате, вывод итоговой суммы по каждому заказу.

Структура проекта

```
Чудо_Обувь/
  main.py
  README.md data/products.json docs/report.md tests/test_main.py
```

Вывод

Освоены базовые конструкции Python и структуры данных. Реализовано приложение «Чудо Обувь» с каталогом, пользователями, корзиной и моделированием заказов.

Занятие 6. Scrum и Kanban. Проект «СпортТовары»

Задача

Освоить принципы Agile, составить Product Backlog, спланировать спринт, создать Kanban-доску.

Product Backlog

Создано 12 User Story. Ключевые (первый спринт):

ID User Story	SP	Приоритет
US-1 Товары с низким остатком	3	Высокий
US-2 Поиск товаров по названию и бренду	5	Высокий
US-3 Аналитика по категориям	5	Средний
US-4 Оформление заказа и сохранение в JSON	8	Высокий
US-5 Журнал действий	3	Средний

Остальные 7 User Story (US-6…US-12): добавление, редактирование, удаление товара, экспорт в CSV, справка, сортировка по цене, подсчёт стоимости склада.

Sprint Backlog

В первый спринт (1 неделя, 6 ак. часов) вошли 5 историй, суммарная оценка — 24 story points.

Kanban-доска

Создана доска с колонками Backlog / To Do / In Progress / Review / Done. WIP-лимит в колонке In Progress — не более 2 задач.

Реализация

Реализованы 4 функции из Sprint Backlog в файле shop.py:

- get_low_stock(products) — товары с количеством ≤ 3;
- search_advanced(products, query, category, min_price, max_price) — комбинированный поиск;
- count_by_category(products) — словарь {категория: количество}; ● save_orders(orders, filename='orders.json') — сохранение заказов в JSON.

Daily Standup и ретроспектива

Проведено 2 стендапа (что сделано / что планируется / препятствия) и ретроспектива (что хорошо / что улучшить / что попробовать в следующем спринте).

Созданные артефакты

Артефакт	Файл

Product Backlog	docs/03_Product_Backlog.md
Sprint Backlog	docs/04_Sprint_Backlog.md
Kanban-доска	docs/05_Kanban_Board.md
Daily Standup	docs/06_Standup_Notes.md
Ретроспектива	docs/07_Retrospective.md
Реализация	shop.py
Журнал, заказы	actions.log, orders.json

Вывод

Освоены принципы Agile и Scrum, составлен Product Backlog из 12 User Story, спланирован спринт, создана Kanban-доска с WIP-лимитом, реализованы 4 функции.

Занятие 7. Git Flow. Ветвление и слияние

Задача

Освоить схему Git Flow, отработать создание и слияние веток, разрешить конфликт, имитировать Pull Request, выполнить hotfix.

Схема ветвления

Ветка	Назначение
main	Стабильная версия
develop	Интеграция новых функций
feature/<название>	Разработка отдельной функции
hotfix/<название>	Срочное исправление

Выполненные задания

1. **Настройка Git Flow:** создана ветка develop, описан файл docs/08_Git_Flow.md.
2. **Feature-ветка:** создана ветка feature/low-stock, реализована get_low_stock, слита в develop.

3. **Конфликт:** созданы две ветки (feature/total-v1 и feature/total-v2) с разными версиями total_sum. При слиянии возник конфликт, разрешён вручную. 4. **Pull Request:** имитирован через docs/10_Pull_Request.md.

5. **Hotfix:** создана ветка hotfix/cart-total, исправлена ошибка в cart_total, слита в main и develop.
6. **Визуализация:** git log --oneline --graph --all сохранён в docs/12_Git_History.md.

Созданные файлы

Файл	Содержание

docs/08_Git_Flow.md Схема ветвления

docs/09_Conflict_Resolution.md	Описание конфликта
docs/10_Pull_Request.md	Имитация PR
docs/11_Hotfix.md	Описание hotfix
docs/12_Git_History.md	Граф коммитов

Вывод

Освоена схема Git Flow с ветками main, develop, feature/*, hotfix/*; отработаны создание и слияние feature-ветки, разрешение конфликта, имитация PR, выполнение hotfix.

Занятие 8. Отладка и тестирование. unittest

Задача

Освоить отладку в VS Code и написать модульные тесты с использованием unittest.

Что сделано

Создана папка tests/ с файлами __init__.py и test_shop.py. Написаны тесты для: ● get_low_stock (3 теста);

- search_advanced (3 теста);
- total_sum (2 теста);
- add_to_cart (3 теста, включая assertRaises);
- корзины (5 тестов).

Отладка в VS Code

Установлена точка останова в функции get_low_stock, запущена отладка через F5. В панели Variables проверены значения переменных products и threshold.

Результаты тестирования

```
python3 -m unittest discover -s tests -v
```

Класс тестов	Кол-во	Что проверяет
TestGetLowStock	3	Пустой список, нет дефицита, один товар
TestSearchAdvanced	3	Поиск по названию, категории, цене
TestTotalSum	2	Пустая корзина, корзина с товарами

TestAddToCart	3	Неверный размер, товар не найден, исключение
TestCart	5	Добавление, увеличение, удаление, изменение, сумма
Итого	**16**	Все прошли успешно

Найденные и исправленные ошибки

№	Ошибка	Исправление
1	get_low_stock обращался к product['quantity'], но у товаров ключ 'sizes' — KeyError cart_total не учитывал количество товара	Введена функция _total_quantity() Исправлено: total = price * quantity

Вывод

Создана структура тестов, написано 16 модульных тестов, использованы основные методы assert, проведена отладка, найдены и исправлены 2 ошибки. Все тесты проходят успешно.

Занятие 9. Интеграция модулей и сборка

Задача

Разделить монолитный файл на модули, настроить импорты, написать интеграционные тесты, создать скрипт сборки build.py.

Структура проекта

```
src/ __init__.py
  catalog.py
  cart.py orders.py analytics.py storage.py tests/
    test_integration.py
```

`main.py`

`build.py`

Модули

Модуль	Функции
catalog.py	get_low_stock, search_advanced, count_by_category, avg_price
cart.py	add_to_cart, remove_from_cart, update_quantity, cart_total
orders.py	create_order, save_orders, load_orders
analytics.py	total_revenue, best_selling_product, average_order_total
storage.py	load_products, save_products

Скрипт сборки build.py

Проверяет версию Python, наличие файла data/products.json, запускает тесты, запускает приложение.

Интеграционные тесты

Написаны тесты полного цикла: добавление в корзину → создание заказа → сохранение → загрузка. Проверено совпадение суммы заказа и суммы корзины.

Результаты сборки

`python build.py`

Этап	Результат
Проверка Python	3.12.3 — OK
Проверка данных	Файл найден — OK
Тесты	16 из 16 — OK

Запуск	Приложение работает

Найденные ошибки

№	Ошибка	Исправление
1	В cart.py ключ 'product' вместо 'name'	Заменено на 'name'
2	create_order не учитывал количество	total = sum(price * quantity)
3	Циклический импорт в cart.py	Убран лишний импорт

Вывод

Модули разделены, импорты настроены, написаны интеграционные тесты, создан скрипт сборки. Все тесты проходят успешно.

Занятие 10. Документация и итоговый релиз

Задача

Завершить проект «СпортТовары», оформить документацию, подготовить презентацию, создать релиз v1.0.

Структура проекта SportShop1

```
SportShop1/ src/__init__.py, catalog.py, cart.py, orders.py, analytics.py,
storage.py tests/ docs/17_API_Reference.md,
18_User_Guide.md,19_Developer_Guide.md,
20_Final_Test_Report.md,21_Presentation.md data/
 main.py
 build.py
 README.md
```

Документация

Файл	Содержание
README.md	Название, описание, установка, запуск, структура, авторы
17_API_Reference.md	Документация всех функций модулей src/
18_User_Guide.md	Руководство пользователя (меню, сценарии)
19_Developer_Guide.md	Руководство разработчика (сборка, тесты, Git)
20_Final_Test_Report.md	Отчёт о тестировании
21_Presentation.md	Презентация (7 слайдов)

Тестирование

```
python3 -m unittest discover -s tests -v
```

Модуль	Тестов Пройдено
catalog.py	5 5
cart.py	4 4
orders.py	3 3
analytics.py	3 3
storage.py	2 2

Итого 17 17

Релиз v1.0

```
gitadd.
gitcommit-m"Day10:финальнаядокументацияирелизv1.0"
git tag -a v1.0 -m "Релиз версии 1.0" git log --oneline
```

Вывод git log:

```
7c9ffe3 (HEAD -> main, tag: v1.0) Day 10: финальная документация и релиз v1.0
```

Вывод

Проект завершён, оформлена документация, все 17 тестов пройдены, создан релиз v1.0, файлы загружены на GitHub.

Заключение

В ходе учебной практики УП.02 (29.09.2026 – 12.10.2026) выполнены все 10 практических занятий, охватывающих полный цикл разработки ПО — от проектирования до релиза.

Результаты:

- спроектирована учебная ОС my_os/StudyOS, реализованы 13 системных вызовов с логированием;
- построена архитектура ИС «Кадровое агентство» (DFD, ERD, структурные карты);
- освоен Git: базовые команды, Git Flow, разрешение конфликтов; ● изучен Python на примере «Чудо Обувь»;
- освоены Scrum и Kanban на проекте «СпортТовары»;
- написаны 16 модульных тестов, проведена отладка в VS Code;
- выполнена интеграция модулей, создан build.py;
- оформлена документация и создан релиз v1.0.

Показатели:

- Все 10 занятий выполнены в полном объёме.
- Все тесты пройдены (17 из 17).
- Создано 5 учебных проектов.

- Репозиторий: https://github.com/Glebas777/GlebTur-UP02

Навыки: работа в VS Code, Python 3.10+, SQLite, Git/GitHub, unittest, Markdown, MS Visio; ООП и структурное проектирование; Agile/Scrum/Kanban; модульное и интеграционное тестирование; оформление технической документации.

Список использованных источников

1. ГОСТ 19.106-78. ЕСПД. Требования к программным документам. 2. ГОСТ 34.602-89. Техническое задание на создание АС.
3. ГОСТ 19.505-79. Руководство оператора.
4. ГОСТ 19.301-79. Программа и методика испытаний.
5. Документация Python 3.10+. URL: https://docs.python.org/3/
6. Документация Git. URL: https://git-scm.com/doc
7. Документация VS Code. URL: https://code.visualstudio.com/docs 8. Документация unittest. URL: https://docs.python.org/3/library/unittest.html
9. Scrum Guide. URL: https://scrumguides.org/
10. Kanban Guide. URL: https://kanbanguides.org/
11. Репозиторий проекта:https://github.com/dlavsegoakkhg-ux/Frolov-Ilya/upload